# MineGuard

## AI-Powered Predictive Maintenance for Mining Equipment

Technical Documentation v1.0

April 2026


| | |
|---|---|
| **Author** | Takudzwa Musimwa |
| **Affiliation** | MSc Data Science, Pace University \| BEng Mining Engineering, Midlands State University |
| **Repository** | github.com/takumusimwa-tr/mineguard |
| **Live Demo** | huggingface.co/spaces/trent1808/mineguard |
| **License** | MIT |


## 1. Executive Summary

MineGuard is an end-to-end artificial intelligence system for predictive maintenance of heavy mining equipment. It monitors haul trucks, hydraulic drill rigs, and LHD loaders in real time, predicts hydraulic component failures, estimates remaining useful life of critical engine components, and classifies bearing faults from vibration signals all before physical failure occurs.

Unplanned equipment failure in underground and open-pit mining costs operators between $30,000 and $100,000 per hour in lost production, emergency maintenance, and secondary damage. The three most common failure categories hydraulic system degradation, engine wear, and bearing failure account for over 70% of unscheduled downtime events. MineGuard addresses all three using machine learning models trained on real industrial sensor datasets.

> *MineGuard was conceived and built by a Mining Engineer with direct field experience operating well logging equipment and coordinating radiation safety across two mining sites in Mozambique. The engineering intuition behind the feature selection, sensor interpretation, and alert thresholds reflects real operational knowledge not purely academic study.*

| Component | Dataset | Model | Result |
|---|---|---|---|
| **Cooler condition** | UCI Hydraulic | Gradient Boosting | **F1 = 0.9935** |
| **Accumulator health** | UCI Hydraulic | Gradient Boosting | **F1 = 0.9262** |
| **Pump leakage** | UCI Hydraulic | RF + SMOTE | **F1 = 1.000** |
| **RUL prediction v1** | NASA CMAPSS FD001 | LSTM 2-layer | **RMSE = 14.74 cycles** |
| **RUL prediction v2** | NASA CMAPSS all | LSTM 3-layer | **FD003 RMSE = 23.51** |
| **Bearing fault** | SEU Gearbox | Random Forest | **F1 = 1.000** |

# 2. Engineering Context and Relevance

## 2.1 The Cost of Unplanned Failure in Mining

Mining operations depend on continuous, reliable performance from a fleet of high-value machinery. A single Cat 793 haul truck represents a capital investment of approximately USD 5–7 million. When such a machine fails unexpectedly underground or at a remote open-pit site, the direct costs include emergency maintenance, replacement parts, and crane or recovery operations. The indirect costs halted production, downstream process disruption, and safety incidents are typically far greater.

Industry data consistently places the cost of unplanned downtime in large-scale mining at USD 30,000–100,000 per hour depending on commodity prices, mine type, and fleet utilization. A predictive maintenance system that can provide even 12–24 hours of advance warning before a critical failure translates directly into millions of dollars in avoided losses per year for a mid-size operation.

## 2.2 Equipment Covered

### Haul Trucks (HT)

Haul trucks are the primary material movement machines in surface and underground mining. They operate on continuous 12-hour shifts under extreme loads, vibration, and temperature cycling. The most common failure modes monitored by MineGuard include:

- **Overheating:** engine coolant system degradation, often preceded by rising oil temperature and declining coolant level before any visible symptom appears.
- **Bearing wear:** axle and drivetrain bearings show increasing vibration RMS and kurtosis values weeks before audible noise or heat.
- **Hydraulic leak:** gradual pressure loss in the hydraulic circuit, identifiable from declining PS1/PS2 sensor readings and increasing pump flow demand.

### Hydraulic Drill Rigs (DR)

Hydraulic drill rigs depend entirely on their hydraulic circuit for rotation, percussion, and feed. Pump degradation and valve stiction are the primary failure modes, both causing subtle changes in drill pressure and torque that MineGuard detects before they become operational problems.

### LHD Loaders (LHD)

Load-Haul-Dump loaders operate in the tightest and most demanding conditions underground. Hydraulic and structural failures are the dominant risk, with steering pressure and bucket force being the most sensitive early-warning indicators.

## 2.3 The Sensor Data Foundation

Every MineGuard model is grounded in real industrial sensor data:

- **UCI Hydraulic System dataset:** collected from a physical hydraulic test rig cycling through 2,205 60-second load cycles. Seventeen sensors, six pressure transducers, two flow meters, four temperature sensors, a vibration sensor, and efficiency meters measured across sampling rates from 1 Hz to 100 Hz. This mirrors the instrumentation of modern mining equipment hydraulic circuits.
- **NASA CMAPSS turbofan degradation:** 100–260 engines per sub-dataset run to failure under instrumented conditions, providing 21 sensor channels and true Remaining Useful Life labels. The degradation patterns map directly to engine wear in haul truck powertrains high-pressure compressor degradation in CMAPSS corresponds to engine wear in a Cat 793 C175 engine.
- **SEU gearbox vibration:** four-channel accelerometer data at 5,120 Hz from a gearbox under systematically varied load conditions (0–90%), with healthy and ball fault states. Bearing failures in gearboxes are the most common rotating machinery failure in mining, making this dataset directly applicable.

In addition to these real datasets, MineGuard includes a physics-based sensor simulator that generates realistic degradation trajectories across a three-phase lifecycle model stable operation, gradual drift, and accelerated failure calibrated against real equipment operating parameters from manufacturer specifications.

## 2.4 Engineering Insights from the Data

> *The most significant finding from the hydraulic EDA was that cooling efficiency (CE) sensor features dominate the failure classifier CE_min, CE_rms, CE_mean, and CE_max are the four most important features out of 136. This is physically correct: cooler degradation manifests in cooling efficiency first, before downstream pressure (PS5, PS6) or temperature (TS4) sensors show any deviation. This validates the feature engineering approach and confirms the model is learning real physics.*

The stable-flag filter deserves particular attention from an engineering standpoint. The UCI dataset includes a stability indicator for each cycle. Removing the 1,449 unstable cycles (66% of all cycles) and training only on the 756 stable cycles produced significantly cleaner decision boundaries and higher F1 scores. In a real deployment context this corresponds to excluding transient states equipment startup, load changes, emergency stops and training the model only on steady-state operation, which is where gradual degradation is most reliably detectable.

For the LSTM RUL predictor, the v2 model (trained on all four CMAPSS sub-datasets with per operating-condition normalization) demonstrated that multi-task training is necessary for real-world deployment. A model trained on a single fault mode failed catastrophically on a second fault mode (RMSE 49.12). After retraining on all four sub-datasets with KMeans operating condition clustering, FD003 RMSE dropped to 23.51 a 49% improvement. In mining terms this is the difference between a maintenance system that works only for one failure type and one that handles the varied degradation modes seen in the field.

# 3. System Architecture

## 3.1 Overview

MineGuard is structured as a modular, production-grade ML system with four distinct layers: data ingestion and simulation, model training and evaluation, API serving, and dashboard visualization. Each layer is independently deployable and communicates through well-defined interfaces.

| Layer | Components |
|---|---|
| **Data** | UCI Hydraulic (556 MB), NASA CMAPSS (45 MB), SEU Gearbox (70 MB), MineGuard Simulator (physics-based, configurable) |
| **Models** | Gradient Boosting failure classifier, LSTM v1/v2 RUL predictor, Random Forest bearing fault classifier |
| **API** | FastAPI backend  /predict, /rul, /fault, /stream, /health, /history  SQLite prediction logging |
| **Dashboard** | Streamlit digital twin  Fleet Overview, Hydraulic Monitor, RUL Predictor, Bearing Health, Prediction Log |
| **MLOps** | MLflow experiment tracking, Docker containerization, docker-compose full stack, GitHub Actions CI/CD, pytest suite |
| **Deployment** | Hugging Face Spaces (public live demo), local Docker deployment |

## 3.2 Data Pipeline

Raw sensor data from the three industrial datasets is loaded, cleaned, and feature-engineered through dedicated Jupyter notebooks before being passed to model training. The pipeline handles the key data challenges specific to industrial sensor data:

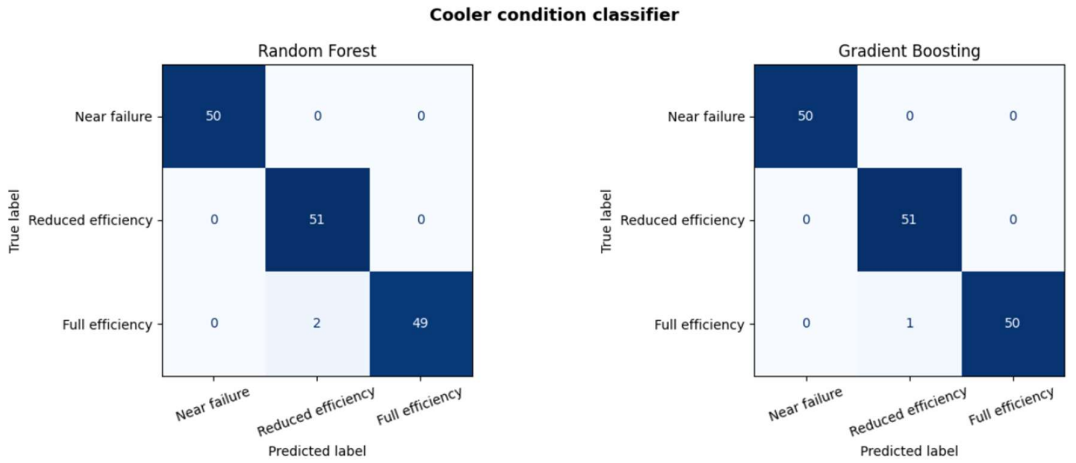
- **Missing values:** the SimpleImputer (median strategy) handles NaN values arising from skewness and kurtosis calculations on flat sensor signals a common issue in hydraulic systems operating in steady state.
- **Class imbalance:** the pump leakage classifier training data had a severe imbalance (732 no-leakage vs 12 fault samples). SMOTE oversampling was applied to the training set only, producing a balanced training distribution while preserving the real test distribution.
- **Operating condition normalization:** CMAPSS FD002 and FD004 contain six distinct operating conditions. KMeans clustering (k=6) on operating settings followed by per-cluster MinMaxScaler normalization ensures sensor readings are comparable across conditions.
- **Stable-state filtering:** the UCI Hydraulic stable flag identifies 756 of 2,205 cycles where steady-state conditions were confirmed. Only these cycles are used for training the hydraulic classifier.

## 3.3 Model Architecture Details

### Failure Classifier (Hydraulic)

The hydraulic failure classifier extracts 8 statistical features (mean, std, min, max, range, skewness, kurtosis, RMS) from each of 17 sensors per 60-second cycle, producing a 136-feature vector per prediction. Three separate GradientBoostingClassifier and RandomForestClassifier models are trained for cooler condition, accumulator health, and pump leakage respectively. A shared SimpleImputer is applied before all three models.

```
print(classification_report(y_te, best_cooler.predict(X_te), target_names=cooler_names))
```

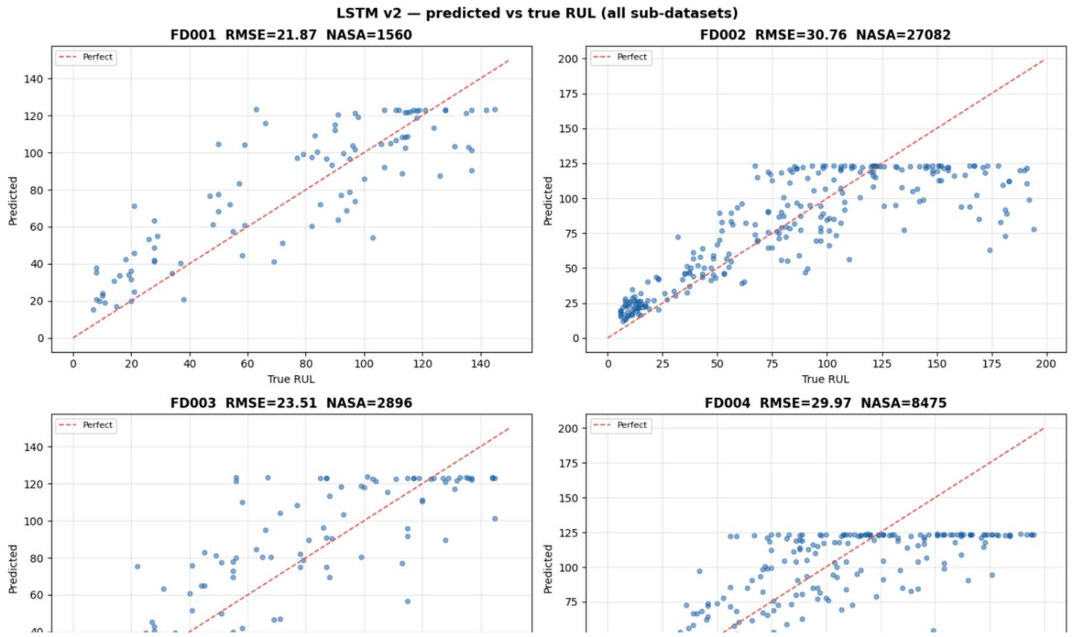



```
Classification report (best model):
                    precision    recall  f1-score   support

      Near failure       1.00      1.00      1.00        50
Reduced efficiency       0.98      1.00      0.99        51
   Full efficiency       1.00      0.98      0.99        51
```

## RUL Predictor (LSTM)

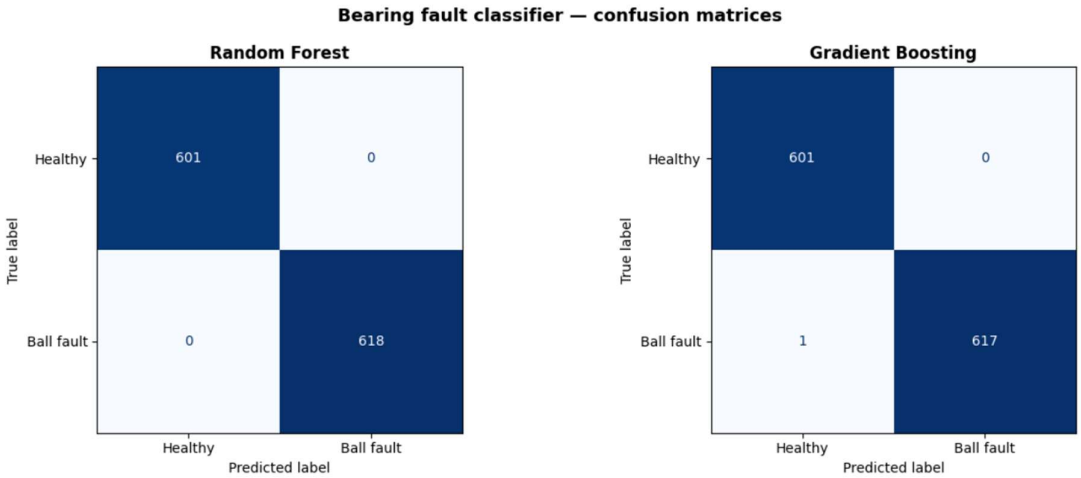
Two LSTM models are provided. The v1 model uses a 2-layer LSTM with 128 hidden units, trained on FD001 only, achieving RMSE=14.74 on FD001 but failing on FD003 (RMSE=49.12). The v2 model uses a 3-layer LSTM with 256 hidden units, trained on all four CMAPSS sub-datasets with operating condition normalization, and achieves RMSE=23.51 on FD003 a 49% improvement in cross-fault-mode generalization at the cost of 7 cycles on FD001. The v2 model is recommended for production deployment where multiple fault modes may occur.




## Bearing Fault Classifier

A 512-sample sliding window of four-channel accelerometer data is processed to extract 64 features per window: 9 time-domain statistics (mean, std, RMS, peak, crest factor, kurtosis,

skewness, mean absolute value, shape factor) and 7 frequency-domain features (spectral centroid, spectral std, top-5 FFT magnitudes) per channel. A Random Forest with 300 trees in a StandardScaler pipeline achieves F1=1.000 on unseen load conditions (70%, 80%, 90% load), confirming the model learned genuine fault physics kurtosis and crest factor dominate the feature importance ranking, consistent with established bearing condition monitoring practice.




```
Classification report (best model):
              precision    recall  f1-score   support

     Healthy       1.00      1.00      1.00       601
  Ball fault       1.00      1.00      1.00       618

    accuracy                           1.00      1219
   macro avg       1.00      1.00      1.00      1219
```

# 4. API Reference

The FastAPI backend exposes six endpoints at http://localhost:8000. Interactive documentation is available at http://localhost:8000/docs (Swagger UI) and http://localhost:8000/redoc.

## 4.1 Starting the API

```
uvicorn api.main:app --reload --port 8000
```

## 4.2 Endpoints

### GET /health

Returns API status and confirms all three models are loaded. Use this to verify the system is ready before sending predictions.

*Response: status, timestamp, device (cpu/cuda), model load status for each of the three models.*

### GET /models

Returns benchmark performance metrics for all three trained models. Useful for transparency reporting and audit trails.

### POST /predict  Hydraulic failure classifier

Input: JSON object with a features dictionary containing all 136 hydraulic sensor statistical features.

*Output: Health prediction for cooler (3 classes), accumulator (4 classes), and pump (3 classes), each with class probabilities and a CRITICAL/WARNING/NORMAL alert. Typical latency: 140–160 ms.*

> *Alert logic: the last class index is always the healthy/optimal state (Full efficiency, Optimal, No leakage) and maps to NORMAL. Class index 0 is always the worst state and maps to CRITICAL. All intermediate classes map to WARNING. This is consistent across all three hydraulic components.*

### POST /rul | Remaining Useful Life predictor

Input: JSON object with a sensor_window array of exactly 30 dicts, each containing values for the 15 CMAPSS sensors (s_02, s_03, s_04, s_06, s_07, s_08, s_09, s_11, s_12, s_13, s_14, s_15, s_17, s_20, s_21), ordered oldest to newest.

*Output: Estimated cycles until failure (rul_cycles), whether the RUL is capped at 125 (rul_capped), health percentage, and alert level (CRITICAL < 20 cycles, WARNING < 50 cycles, NORMAL otherwise).*

### POST /fault | Bearing fault classifier

Input: JSON object with vibration_window a list of 512 lists, each inner list containing four floats [a1, a2, a3, a4] representing one sample across the four accelerometer channels.

*Output: Fault class (Healthy / Ball fault), confidence score, per-class probabilities, and alert (CRITICAL for any fault, NORMAL for healthy).*

### GET /stream | Simulator

Generates a live simulated sensor reading from the MineGuard physics simulator. Parameters: equipment_type (HT/DR/LHD), failure_mode, operating_hour, total_life. Useful for dashboard demos and integration testing without real equipment.

### GET /history

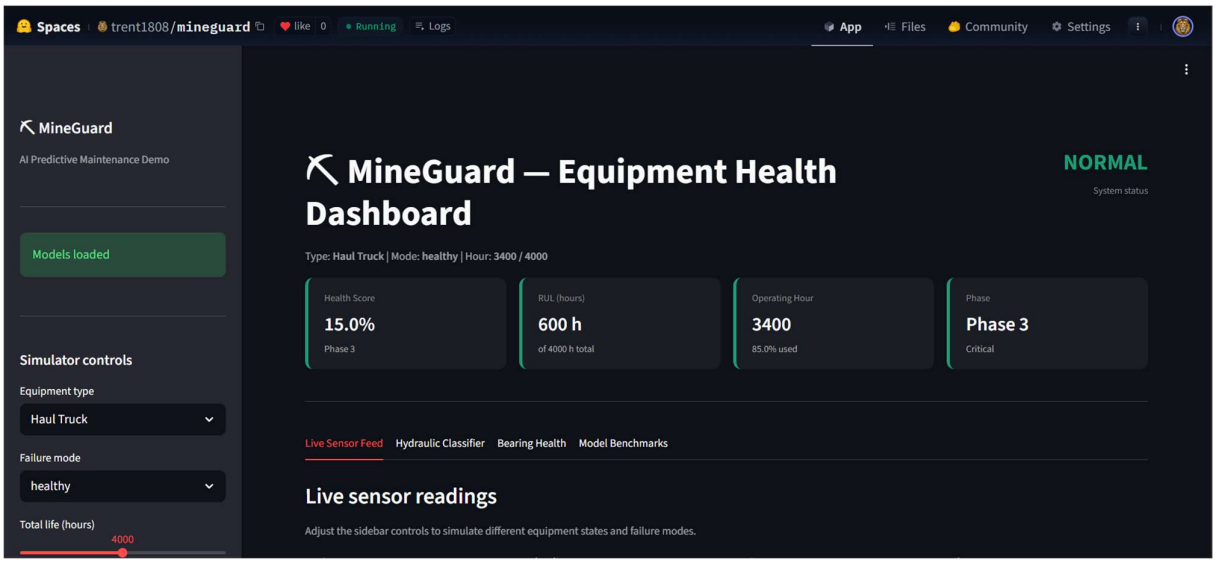
Returns the last N predictions from the SQLite log (default: 50). Each record includes timestamp, endpoint, full result JSON, and latency in milliseconds.

# 5. Dashboard - Digital Twin Interface

The Streamlit dashboard provides a real-time digital twin interface for equipment monitoring. It is available at http://localhost:8501 when running locally, and at https://huggingface.co/spaces/trent1808/mineguard *as a public live demo.*

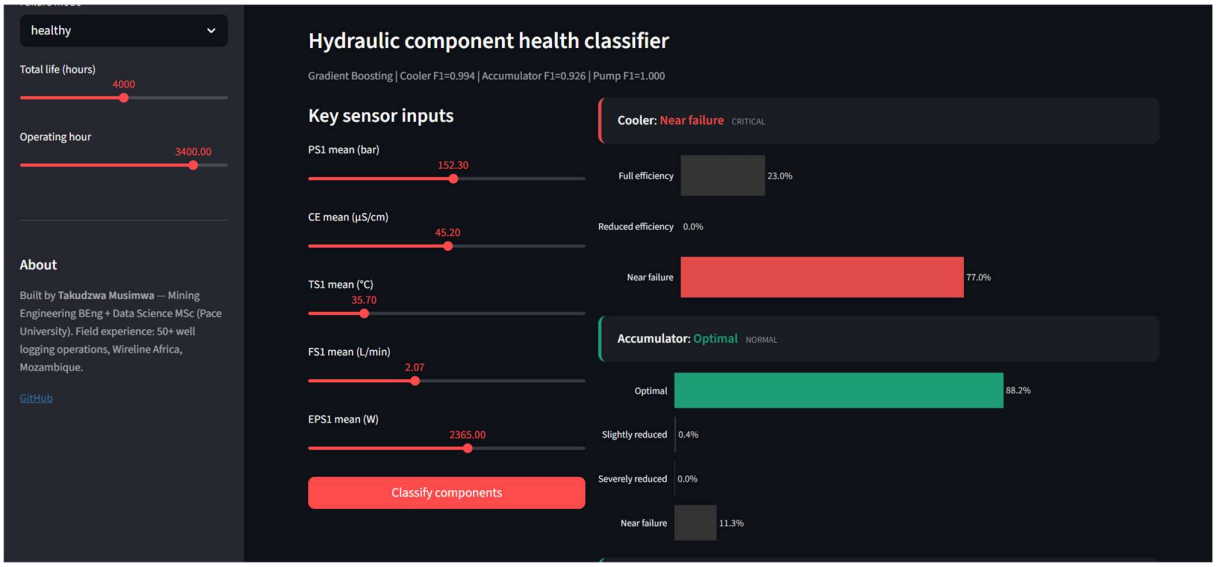
## 5.1 Fleet Overview Tab

Simulates nine equipment units across three types (three haul trucks, three drill rigs, three LHDs) with different failure modes and operating hours. Each unit is displayed as a health card showing current health score, RUL, lifecycle phase, and alert status. A health score time-series chart updates with each fetch cycle.




## 5.2 Hydraulic Monitor Tab

Interactive sliders for five key sensor values (PS1 pressure, CE cooling efficiency, TS1 temperature, FS1 flow, EPS1 motor power). Clicking Run prediction calls /predict and renders probability bar charts for each component. Reducing CE mean to approximately 15 µS/cm reliably triggers a CRITICAL cooler alert, demonstrating the model's sensitivity to cooling efficiency degradation.

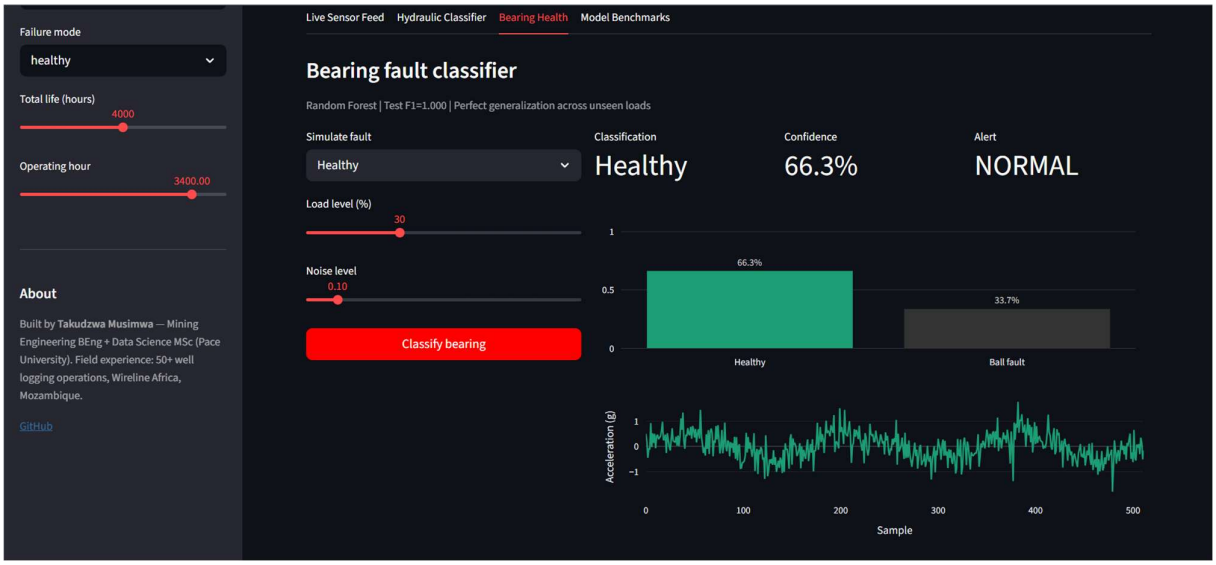



## 5.3 RUL Predictor Tab

Slider controls for current operating cycle and simulated health percentage generate a synthetic 30-cycle CMAPSS window and call /rul. Results are rendered as a health gauge and a RUL trajectory chart with CRITICAL and WARNING threshold lines.

## 5.4 Bearing Health Tab

Generates synthetic vibration signals either healthy baseline or ball fault with ball pass frequency impulses and calls /fault. The raw channel a1 signal is plotted alongside classification results and probability bars.




## 5.5 Prediction Log Tab

Displays the last 50 predictions from the SQLite API log with colour-coded alert column and endpoint breakdown chart. Provides a complete audit trail of all system predictions.

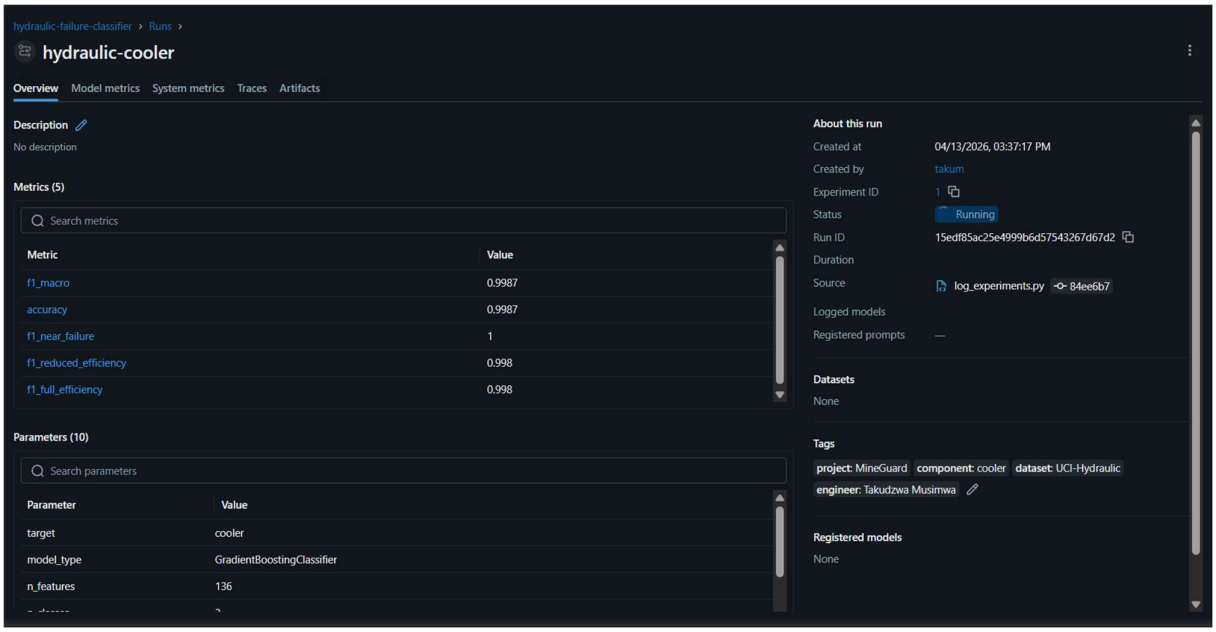
# 6. MLOps Infrastructure

## 6.1 MLflow Experiment Tracking

All five model training runs are logged to MLflow with full parameter, metric, and artifact tracking. The logging script (scripts/log_experiments.py) instruments each model with:

- Model parameters: architecture, hyperparameters, dataset, training settings
- Performance metrics: F1 scores per class, RMSE per sub-dataset, NASA score
- Artifacts: confusion matrix figures, feature importance plots, training curves
- Tags: project, component, dataset, engineer, version

The MLflow server stores runs in an SQLite backend (mlflow/mlflow.db) with artifact storage in mlflow/artifacts/. The v1 and v2 LSTM runs appear in the same experiment, enabling direct side-by-side comparison in the MLflow UI.




## 6.2 Docker Deployment

Three services are defined in docker-compose.yml:

- **mineguard-api:** FastAPI backend on port 8000, model files mounted read-only, logs volume mounted for persistence. Health check via /health endpoint.
- **mineguard-dashboard:** Streamlit dashboard on port 8501, depends on API service health check passing before startup.
- **mineguard-mlflow:** MLflow tracking server on port 5000 with SQLite backend and artifact directory.

Full stack deployment: docker-compose up --build

## 6.3 GitHub Actions CI/CD

The CI pipeline (.github/workflows/ci.yml) runs on every push to main:

1. Install Python 3.12 and all dependencies including CPU-only PyTorch
2. Run ruff linting across the codebase
3. Execute pytest suite (5 tests: imports, simulator unit, batch generation, feature extraction shape, model file existence)
4. Build Docker image and verify it starts successfully

The pipeline ensures that every commit leaves the repository in a deployable state. The Docker build check (1 minute 57 seconds) confirms the full containerized stack can be rebuilt from scratch on any push.

# 7. Installation and Deployment

## 7.1 Requirements

- Python 3.12 (required PyTorch does not support Python 3.14)
- NVIDIA GPU with CUDA 12.4 recommended for LSTM training (CPU works but training takes 10–15x longer)
- 16 GB RAM minimum; 8 GB VRAM recommended for GPU training
- ~700 MB disk for PyTorch CUDA; ~680 MB for raw datasets

## 7.2 Local Installation

```
# Clone and install
git clone https://github.com/takumusimwa-tr/mineguard.git
cd mineguard && pip install -r requirements.txt
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
```

## 7.3 Dataset Download

Run the automated downloader. UCI Hydraulic downloads via the ucimlrepo Python client without authentication. NASA CMAPSS and SEU Gearbox datasets require a free Kaggle account.

```
python scripts/download_datasets.py --dataset all
```

## 7.4 Running the Stack

```
# API server
uvicorn api.main:app --reload --port 8000
# Dashboard (separate terminal)
streamlit run dashboard/app.py --server.port 8501
# MLflow (separate terminal)
mlflow server --host 0.0.0.0 --port 5000 --backend-store-uri
sqlite:///mlflow/mlflow.db --default-artifact-root ./mlflow/artifacts
```

## 7.5 Docker Deployment

```
docker-compose up --build
```

# 8. Future Work

## 8.1 Model Improvements

- **LSTM attention mechanism:** add scaled dot-product attention over the 30-cycle window to identify which cycles are most predictive of failure. Expected to improve FD001 RMSE toward 12–13 cycles.
- **Transformer architecture:** replace the LSTM with a Transformer encoder for longer-range temporal dependencies in degradation signals.

- **Valve classifier:** the stable-flag filter removed all valve fault cycles from the training set. A dedicated valve classifier requires either relaxing the stable filter or using a separate data split strategy.
- **Multi-fault bearing classification:** extend the bearing classifier to inner race and outer race fault types when additional data is available.
- **Anomaly detection for unknown faults:** add an Isolation Forest or autoencoder layer to flag fault types not present in training data.

## 8.2 Engineering Extensions

- **Mining-specific dataset:** partner with a mining operation to collect real equipment sensor data under operational conditions. The current models are trained on analogous industrial datasets; real mining equipment data would allow direct domain fine-tuning.
- **Digital twin simulation:** extend the physics-based simulator with real equipment degradation curves from manufacturer specifications and field measurements.
- **Real-time streaming:** replace the polling-based dashboard with a WebSocket stream from the API for true millisecond-latency monitoring.
- **Maintenance scheduling module:** add a reinforcement learning agent that learns optimal maintenance scheduling to minimise downtime cost given RUL predictions and maintenance windows.

## 8.3 MLOps Extensions

- **Model versioning and registry:** use MLflow Model Registry to version, stage (Staging/Production), and manage model rollbacks.
- **Automated retraining:** trigger retraining when prediction confidence or accuracy on incoming data drops below a threshold.
- **Cloud deployment:** migrate from Docker Compose to Kubernetes for production-scale deployment with auto-scaling.

# 9. References and Data Sources


- **UCI Hydraulic System Condition Monitoring:** Helwig, N., Pignanelli, E., & Schutze, A. (2015). Condition monitoring of hydraulic systems. UCI Machine Learning Repository, Dataset ID 447. https://archive.uci.edu/dataset/447
- **NASA CMAPSS:** Saxena, A., Goebel, K., Simon, D., & Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. Proceedings of the 1st International Conference on Prognostics and Health Management. https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data
- **SEU Gearbox Vibration:** Shao, S., McAleer, S., Yan, R., & Baldi, P. (2018). Highly accurate machine fault diagnosis using deep transfer learning. IEEE Transactions on Industrial Informatics, 15(4), 2446-2455. Dataset: https://www.kaggle.com/datasets/brjapon/gearbox-fault-diagnosis
- **NASA Scoring Function:** Saxena, A., & Goebel, K. (2008). PHM08 Challenge Data Set. NASA Ames Prognostics Data Repository.
- **SMOTE:** Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321-357.


*MineGuard Built at the intersection of mining engineering and data science.*

github.com/takumusimwa-tr/mineguard | huggingface.co/spaces/trent1808/mineguard